

DSA JAVA PRACTICE NOTES

Sessions 1–5

Topics: Arrays • HashMap • ArrayList • Linked List • Queue • Stack • Two Pointers • Binary Search

SESSION 1 – ARRAYS & HASHMAP

Problem 1: Running Sum of 1D Array

Platform: LeetCode

Problem: 1480. Running Sum of 1D Array

Aim

To calculate the running sum of an array, where each element represents the sum of all elements from the beginning of the array up to that position.

Input

```
nums = [1, 2, 3, 4]
```

Output

```
[1, 3, 6, 10]
```

Java Code

```
class Solution {  
    public int[] runningSum(int[] nums) {  
  
        for (int i = 1; i < nums.length; i++) {  
            nums[i] = nums[i] + nums[i - 1];  
        }  
  
        return nums;  
    }  
}
```

```
}  
}
```

Explanation

- First element = 1
- Second element = 1 + 2 = 3
- Third element = 1 + 2 + 3 = 6
- Fourth element = 1 + 2 + 3 + 4 = 10

Therefore:

[1, 3, 6, 10]

Complexity

Time: O(n)

Space: O(1)

Problem 2: Phone Book

Platform: HackerRank

Problem: Phone Book

Aim

To store names and phone numbers using a **HashMap** and retrieve the phone number for a given person's name.

Input

```
3  
uncle sam  
99912222  
tom  
11122222
```

harry
12299933
uncle sam
uncle tom
harry

Output

uncle sam=99912222
Not found
harry=12299933

Java Code

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine());

        HashMap<String, String> phoneBook = new HashMap<>();

        for (int i = 0; i < n; i++) {

            String name = sc.nextLine();
            String phone = sc.nextLine();

            phoneBook.put(name, phone);
        }

        while (sc.hasNextLine()) {

            String query = sc.nextLine();

            if (phoneBook.containsKey(query)) {
                System.out.println(query + "=" + phoneBook.get(query));
            } else {
                System.out.println("Not found");
            }
        }
    }
}
```

```
}  
  
    sc.close();  
}  
}
```

Complexity

Time: $O(n + q)$ average case

Space: $O(n)$

SESSION 2 – ARRAYLIST BASICS & OPERATIONS

Problem 3: Find All Numbers Disappeared in an Array

Platform: LeetCode

Problem: 448. Find All Numbers Disappeared in an Array

Aim

To find all numbers in the range 1 to n that do not appear in the given array.

Input

`nums = [4, 3, 2, 7, 8, 2, 3, 1]`

Output

`[5, 6]`

Java Code

```
import java.util.*;
```

```
class Solution {
```

```
    public List<Integer> findDisappearedNumbers(int[] nums) {
```

```

List<Integer> result = new ArrayList<>();

for (int i = 0; i < nums.length; i++) {

    int index = Math.abs(nums[i]) - 1;

    if (nums[index] > 0) {
        nums[index] = -nums[index];
    }
}

for (int i = 0; i < nums.length; i++) {

    if (nums[i] > 0) {
        result.add(i + 1);
    }
}

return result;
}
}

```

Explanation

The array itself is used to mark which numbers exist.

For every value, its corresponding index is made negative. After marking, positions that remain positive represent missing numbers.

Complexity

Time: $O(n)$

Space: $O(1)$, excluding the returned list

Problem 4: Sort Colors

Platform: LeetCode

Problem: 75. Sort Colors

Aim

To sort an array containing only 0, 1, and 2 in-place without using the built-in sorting function.

Input

```
nums = [2,0,2,1,1,0]
```

Output

```
[0,0,1,1,2,2]
```

Approach

Use three pointers:

- **low** — position for zero
- **mid** — current element
- **high** — position for two

This is known as the **Dutch National Flag algorithm**.

Java Code

```
class Solution {  
  
    public void sortColors(int[] nums) {  
  
        int low = 0;  
        int mid = 0;  
        int high = nums.length - 1;  
  
        while (mid <= high) {  
  
            if (nums[mid] == 0) {  
  
                swap(nums, low, mid);  
  
                low++;  
                mid++;  
  
            } else if (nums[mid] == 1) {
```

```

        mid++;

    } else {

        swap(nums, mid, high);

        high--;
    }
}

private void swap(int[] nums, int i, int j) {

    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
}

```

Complexity

Time: $O(n)$

Space: $O(1)$

SESSION 3 – LINKED LIST, DEQUE & QUEUE

Problem 5: Design Linked List

Platform: LeetCode

Problem: 707. Design Linked List

Aim

To implement a singly linked list with operations such as getting, adding, and deleting elements.

Sample Input

```

["MyLinkedList",
 "addAtHead",
 "addAtTail",

```

```
"addAtIndex",  
"get",  
"deleteAtIndex",  
"get"]
```

```
[],  
[1],  
[3],  
[1,2],  
[1],  
[1],  
[1]]
```

Sample Output

```
[null,null,null,null,2,null,3]
```

Java Code

```
class MyLinkedList {  
  
    class Node {  
  
        int val;  
        Node next;  
  
        Node(int val) {  
            this.val = val;  
        }  
    }  
  
    Node head;  
    int size;  
  
    public MyLinkedList() {  
        head = null;  
        size = 0;  
    }  
  
    public int get(int index) {  
  
        if (index < 0 || index >= size) {  
            return -1;  
        }  
    }  
}
```

```

Node curr = head;

for (int i = 0; i < index; i++) {
    curr = curr.next;
}

return curr.val;
}

public void addAtHead(int val) {

    Node newNode = new Node(val);

    newNode.next = head;
    head = newNode;

    size++;
}

public void addAtTail(int val) {

    Node newNode = new Node(val);

    if (head == null) {
        head = newNode;
        size++;
        return;
    }

    Node curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    curr.next = newNode;

    size++;
}

public void addAtIndex(int index, int val) {

    if (index < 0 || index > size) {

```

```

        return;
    }

    if (index == 0) {
        addAtHead(val);
        return;
    }

    Node curr = head;

    for (int i = 0; i < index - 1; i++) {
        curr = curr.next;
    }

    Node newNode = new Node(val);

    newNode.next = curr.next;
    curr.next = newNode;

    size++;
}

public void deleteAtIndex(int index) {

    if (index < 0 || index >= size) {
        return;
    }

    if (index == 0) {

        head = head.next;
        size--;

        return;
    }

    Node curr = head;

    for (int i = 0; i < index - 1; i++) {
        curr = curr.next;
    }

    curr.next = curr.next.next;

```

```
    size--;  
  }  
}
```

Complexity

Time: $O(n)$

Space: $O(n)$

Problem 6: Implement Queue Using Stacks

Platform: LeetCode

Problem: 232. Implement Queue using Stacks

Aim

To implement a FIFO queue using two stacks.

Sample Input

["MyQueue", "push", "push", "peek", "pop", "empty"]

[[], [1], [2], [], [], []]

Sample Output

[null, null, null, 1, 1, false]

Approach

Use two stacks:

- **input** stores newly inserted elements.
- **output** provides elements in queue order.
- When **output** is empty, move all elements from **input** to **output**.

Java Code

```
import java.util.Stack;

class MyQueue {

    Stack<Integer> input;
    Stack<Integer> output;

    public MyQueue() {

        input = new Stack<>();
        output = new Stack<>();
    }

    public void push(int x) {

        input.push(x);
    }

    public int pop() {

        moveElements();

        return output.pop();
    }

    public int peek() {

        moveElements();

        return output.peek();
    }

    public boolean empty() {

        return input.isEmpty() && output.isEmpty();
    }

    private void moveElements() {

        if (output.isEmpty()) {

            while (!input.isEmpty()) {
                output.push(input.pop());
            }
        }
    }
}
```

```
}  
}  
}
```

Complexity

- **Push:** $O(1)$
 - **Pop:** Amortized $O(1)$
 - **Peek:** Amortized $O(1)$
 - **Space:** $O(n)$
-

SESSION 4 – STACK

Problem 7: Valid Parentheses

Platform: LeetCode

Problem: 20. Valid Parentheses

Aim

To determine whether the brackets in a string are correctly opened and closed using a Stack.

Input

```
s = "()[]{}"
```

Output

```
true
```

Approach

When an opening bracket is found, push the expected closing bracket onto the stack.

When a closing bracket is found, compare it with the bracket at the top of the stack.

Java Code

```
import java.util.Stack;

class Solution {

    public boolean isValid(String s) {

        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {

            if (ch == '(') {
                stack.push(')');
            }
            else if (ch == '{') {
                stack.push('}');
            }
            else if (ch == '[') {
                stack.push(']');
            }
            else {

                if (stack.isEmpty() || stack.pop() != ch) {
                    return false;
                }
            }
        }

        return stack.isEmpty();
    }
}
```

Example 2

Input: `s = "([)]"`

Output: `false`

Example 3

Input: `s = "([])"`

Output: `true`

Complexity

Time: $O(n)$

Space: $O(n)$

Problem 8: Min Stack

Platform: LeetCode

Problem: 155. Min Stack

Aim

To design a stack that supports **push**, **pop**, **top**, and retrieving the minimum value in $O(1)$ time.

Sample Input

```
["MinStack",  
 "push",  
 "push",  
 "push",  
 "getMin",  
 "pop",  
 "top",  
 "getMin"]
```

```
[[],  
 [-2],  
 [0],  
 [-3],  
 [],  
 [],  
 [],  
 []]
```

Sample Output

```
[null,null,null,null,-3,null,0,-2]
```

Approach

Use two stacks:

- `stack` stores all values.
- `minStack` stores the minimum values.

Java Code

```
import java.util.Stack;

class MinStack {

    Stack<Integer> stack;
    Stack<Integer> minStack;

    public MinStack() {

        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {

        stack.push(val);

        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {

        int value = stack.pop();

        if (value == minStack.peek()) {
            minStack.pop();
        }
    }

    public int top() {

        return stack.peek();
    }

    public int getMin() {
```

```
    return minStack.peek();  
  }  
}
```

Complexity

- **Push:** $O(1)$
 - **Pop:** $O(1)$
 - **Top:** $O(1)$
 - **GetMin:** $O(1)$
 - **Space:** $O(n)$
-

SESSION 5 – ARRAYS UTILITY & BINARY SEARCH

Problem 9: Merge Sorted Array

Platform: LeetCode

Problem: 88. Merge Sorted Array

Aim

To merge two sorted arrays into `nums1` in sorted order using two pointers.

Input

`nums1 = [1,2,3,0,0,0]`
`m = 3`

`nums2 = [2,5,6]`
`n = 3`

Output

[1,2,2,3,5,6]

Approach

Start from the end of both arrays.

Use three pointers:

- `i` — last valid element in `nums1`
- `j` — last element in `nums2`
- `k` — last available position in `nums1`

Place the larger element at position `k`.

Java Code

```
class Solution {  
  
    public void merge(int[] nums1, int m, int[] nums2, int n) {  
  
        int i = m - 1;  
        int j = n - 1;  
        int k = m + n - 1;  
  
        while (i >= 0 && j >= 0) {  
  
            if (nums1[i] > nums2[j]) {  
  
                nums1[k] = nums1[i];  
                i--;  
  
            } else {  
  
                nums1[k] = nums2[j];  
                j--;  
            }  
  
            k--;  
        }  
    }  
}
```

```
while (j >= 0) {  
    nums1[k] = nums2[j];  
    j--;  
    k--;  
}  
}
```

Complexity

Time: $O(m + n)$

Space: $O(1)$

Problem 10: Find Smallest Letter Greater Than Target

Platform: LeetCode

Problem: 744. Find Smallest Letter Greater Than Target

Aim

To find the smallest character that is lexicographically greater than the target using binary search.

Input

```
letters = ['c', 'f', 'j']  
target = 'a'
```

Output

```
'c'
```

Approach

Use binary search to find the first character strictly greater than the target.

If no character is greater than the target, return the first character using:

```
letters[left % letters.length]
```

Java Code

```
class Solution {  
  
    public char nextGreatestLetter(char[] letters, char target) {  
  
        int left = 0;  
        int right = letters.length - 1;  
  
        while (left <= right) {  
  
            int mid = left + (right - left) / 2;  
  
            if (letters[mid] <= target) {  
  
                left = mid + 1;  
  
            } else {  
  
                right = mid - 1;  
            }  
        }  
  
        return letters[left % letters.length];  
    }  
}
```

Example 2

Input:

```
letters = ['c', 'f', 'j']  
target = 'c'
```

Output:

```
'f'
```

Example 3

Input:

```
letters = ['x', 'x', 'y', 'y']  
target = 'z'
```

Output:

'x'

Complexity

Time: $O(\log n)$

Space: $O(1)$
